The Journal of Systems and Software 222 (2025) 112352

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Enhancing concurrency vulnerability detection through AST-based static fuzz mutation☆

Wei Zheng [a], Chang Liu [a], Peiran Deng [a], Xiang Chen [b], Xiaoxue Wu [c,*]

[a] Northwestern Polytechnical University, School of Software, Xi'an, 710100, China
[b] Nantong University, School of Artificial Intelligence and Computer Science, Nantong, 226000, China
[c] Yangzhou University, School of Information Engineering, Yangzhou, 225000, China

ARTICLE INFO

Keywords:
Concurrent programs
Mutation testing
Abstract Syntax Tree
Static fuzz mutation
Go language

ABSTRACT

As multi-threaded and highly concurrent programs are increasingly used, their inherent uncertainty significantly impacts program stability. Traditional testing methods often struggle to effectively detect specific concurrency vulnerabilities because these vulnerabilities are triggered only under particular circumstances, making detection at the vulnerability-triggering level challenging. In view of this, we propose a static fuzz mutation testing method based on Abstract Syntax Tree (AST). This method leverages the fine-grained granularity of ASTs to optimize test suites for concurrency vulnerabilities detection. Initially, We analyze and classify the concurrency vulnerabilities found in Go source code, and generate vulnerability feature mutation operators (mutation operators with concurrency vulnerability feature). Next, we propose static fuzz mutation method and heuristic algorithms at the AST level and apply them to mutators. Ultimately, we screened over 200 code slices from 8 open-source projects for static fuzz mutation testing. The results indicate that introducing vulnerability feature mutation operators improved the number of mutant by approximately 22.15% across various types of concurrency vulnerability samples. This enhancement elevated the probability of triggering concurrency vulnerabilities in the program. After incorporating data from the Go language standard library, experiments further confirmed that our proposed static fuzz mutation testing method can effectively improve the accuracy of concurrency vulnerability detection.

1. Introduction

Concurrent programming has gained significant popularity in the era of multi-core processors, as it effectively leverages hardware resources to enhance software performance. With the increasing adoption of concurrency in applications to enhance efficiency, the prevalence of concurrency-related issues has become more pronounced (Cao et al., 2023b). However, compared to sequential programs, concurrent programs are more susceptible to severe software failures. One major challenge is the non-deterministic interleaving of threads, which can lead to concurrency bugs such as atomicity violations, data races, and more. The inherent unpredictability of thread scheduling complicates the reproduction of errors or anomalies, leading to significant safety risks (Jian et al., 2018). Additionally, certain input conditions or thread execution patterns may trigger concurrency bugs, leading to system failures such as memory corruption, assertion violations, and crashes (Tu et al., 2019). As a result, effectively detecting these concurrency bugs is of paramount importance.

The continuous iteration and widespread adoption of new technologies present both opportunities and challenges, inevitably introducing new software vulnerabilities. The increasing prevalence of concurrent programs has exacerbated this issue, as the uncertainty and complexity of runtime behaviors make testing more difficult. In concurrent environments, the interference and interaction between threads can lead to numerous hidden vulnerabilities, complicating the task of ensuring program correctness and stability. These vulnerabilities often manifest only under specific execution timelines, placing higher demands on the testing and feature extraction processes for concurrency vulnerabilities. Over the past decade, significant research efforts have focused on developing methods to detect these concurrency vulnerabilities (Jia and Harman, 2010; Parihar and Bharti, 2019), making it a prominent and challenging area of current research. In this context, vulnerability detection in concurrent environments has become a shared challenge for both testing engineers and researchers. Addressing this issue requires continuous innovation and exploration of new methods and

☆ Editor: W. Eric Wong.

* Corresponding author.
E-mail address: xiaoxuewu@yzu.edu.cn (X. Wu).

https://doi.org/10.1016/j.jss.2025.112352
Received 3 September 2024; Received in revised form 5 January 2025; Accepted 16 January 2025
Available online 23 January 2025

technologies to ensure the reliability and security of software systems.

In previous studies, researchers have proposed different methods to identify concurrency vulnerabilities. Dynamic detection tools, such as those utilizing fuzz testing, alongside data flow and execution path analysis tools grounded in program primitives, are both expensive and challenging to maintain. Fuzz testing typically suffers from a low likelihood of triggering concurrency vulnerabilities, resulting in inefficient detection (Cao et al., 2023a). Similarly, static detection methods that rely on data flow and path analysis may work well for certain traditional programming languages but often fail to deliver effective results when applied to languages with distinct multithreading mechanisms (Inverso et al., 2015; Kroening et al., 2016).

Static analysis, a code analysis technique that does not depend on the actual execution of a program, also incurs substantial costs and faces significant challenges in maintenance and optimization (Park et al., 2021). It involves the automatic examination of program source code or bytecode to detect potential errors and security vulnerabilities. However, in the context of concurrent programming, static analysis often encounters inherent limitations that hinder its efficiency and accuracy in vulnerability detection (Gosain and Sharma, 2015).

In software testing, mutation testing and static fuzz mutation are two prominent techniques used to enhance test coverage and identify potential vulnerabilities. Mutation testing involves systematically introducing small changes, or mutations, into the code to create mutant. The effectiveness of existing test cases is then assessed based on their ability to detect these mutants, providing a measure of test quality and coverage (Papadakis et al., 2019; Wong, 2001). This method is dynamic, requiring the execution of test cases against the mutants, and is highly valued for its capacity to reveal weaknesses in test suites and improve their robustness (Zhang et al., 2016). In contrast, static fuzz mutation is a more recent approach that combines static analysis with fuzz testing techniques. Unlike traditional mutation testing, which relies on running the program, static fuzz mutation operates at the code or Abstract Syntax Tree (AST) level without executing the program (Lemieux and Sen, 2018). This method generates mutant by introducing changes directly into the program's static structure and then applying fuzz testing to these mutants. This approach enables early detection of potential vulnerabilities and enhances the accuracy of vulnerability detection by leveraging static analysis to identify and mutate potential points of failure before execution. By integrating static analysis with fuzz testing, static fuzz mutation offers a complementary strategy to dynamic mutation testing, broadening the scope of vulnerability discovery and improving overall test coverage (Shastry et al., 2017).

This paper proposes a testing method that integrates static fuzz mutation with mutation testing to address the challenges of detecting concurrency vulnerabilities. We extend existing static fuzz mutation technology and design a sequence of mutation operators tailored to concurrency vulnerability characteristics. The goal is to enhance both the efficiency and accuracy of concurrency vulnerability detection and to address the research gap in detecting such vulnerabilities specifically within the Go language.

The Go language, a statically typed concurrent programming language, was designed to provide a simple, efficient, and secure approach to building multi-threaded software. As concurrent programming languages play an increasingly vital role in software development across various industries, Go introduces mechanisms like message passing to reduce concurrency vulnerabilities. However, research by Ray et al. (2014) suggests that these methods have not significantly reduced the prevalence of concurrency vulnerabilities. Their study highlights that such vulnerabilities in Go programs are more commonly triggered by improper use of channels rather than shared memory misuse. Table 1 presents statistical data from various sources (including Docker, Kubernetes fix submissions, CWE, and others) on concurrency vulnerabilities across major programming languages. Although these statistics

Table 1
Statistical results from various sources on concurrency vulnerabilities in major languages (Ray et al., 2014; Li et al., 2019a).

	C/C++	C#	Java
Data competition	52.2%	77.7%	65.4%
Deadlock	34.5%	14.4%	17.1%
Shared memory operation	23.4%	9.4%	9.2%
Messaging interface	1.1%	2.1%	7.3%

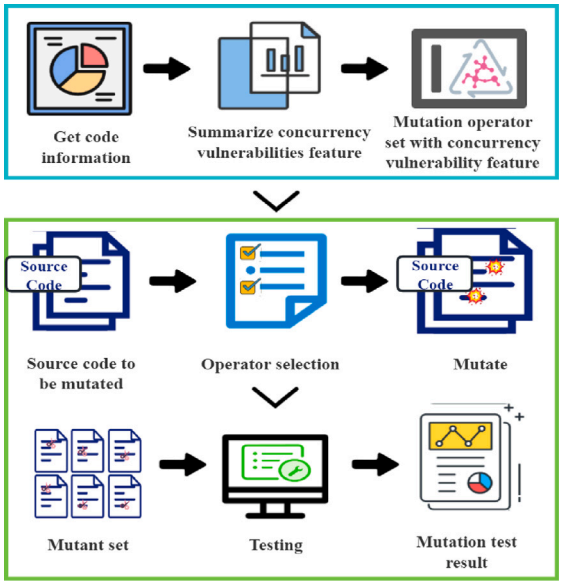

Fig. 1. Workflow of our approach. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

are temporally limited, they indicate that while Go incorporates specialized synchronization mechanisms to address certain concurrency vulnerabilities, these efforts have not significantly reduced their overall occurrence.

Our overall work is illustrated in Fig. 1. Firstly, as shown in the blue framework in Fig. 1, we design vulnerability feature mutation operators based on the feature of concurrency vulnerabilities to generate mutants closer to real defect code in subsequent mutations. Secondly, as depicted in the green framework in Fig. 1, we developed a static fuzz mutation testing system with static fuzzing mutation methods, which combined adaptive mutation operator selection and heuristics algorithms to continuously enhance the effectiveness of mutation testing. In view of the above two aspects of work, we proposed two research questions:

- **RQ1: Are vulnerability feature operators more effective in generating samples with concurrency vulnerabilities compared to traditional mutation operators that do not consider concurrency features?**
- **RQ2: Do adaptive mutation operator selection and heuristic algorithms offer superior performance compared to traditional random mutators?**

We assessed the effectiveness of the mutation operator sequences by examining publicly disclosed concurrency vulnerabilities in cross-version code segments from large Go projects. We adjusted operator weights in these sequences using algorithms and then applied the optimized mutation operator sequences as mutators in mutation testing. Mutation testing was conducted on open-source projects, followed by feedback adjustments to the mutation operator sequences. Finally, we compared the detection results with the multithreaded vulnerabilities disclosed in these open-source projects to evaluate their effectiveness.

To address our research questions, we divided our experimental source code selection into two parts. First, we collected code from the Go language standard library, selecting files based on the presence of corresponding test files with a "_test.go" extension for each ".go" file in each second-level branch of the golang/go repository. For the two large open-source projects, we selected code slices with more than 15 lines. We performed comparative experiments on vulnerability feature operators and static fuzz mutation method from various perspectives, using data from different sources.

2. Background and related work

Concurrent programs are inherently more prone to severe software malfunctions compared to their sequential counterparts. First, the unpredictable nature of thread interleaving in concurrent programs can give rise to vulnerabilities such as atomicity violations and data races. This unpredictability not only complicates the reproduction of anomalous results or vulnerabilities but also presents significant safety challenges (Chen et al., 2018). Second, specific input scenarios and thread execution sequences can trigger concurrency-related issues, potentially leading to critical system failures such as memory corruption and assertion errors (Cai et al., 2019b; Zheng et al., 2022; Ren et al., 2021). As a result, the ability to accurately and effectively identify these concurrency-related vulnerabilities is of utmost importance.

In the rest of this section, we introduce the background of concurrency vulnerability detection techniques and methods, as well as the methods needed for experiments.

2.1. Static analysis and dynamic detection of concurrency vulnerabilities detection

Extensive research (Godefroid, 2007; Dam et al., 2018; Yang et al., 2022) has been dedicated to identifying errors in concurrent software. Static analysis techniques (Barbar and Sui, 2021) analyze context-sensitive relationships within programs to detect potential issues without requiring actual execution. However, these methods often produce a significant number of false positives to ensure comprehensive coverage, necessitating additional validation to confirm the detected issues. In contrast, dynamic detection methods (Ahmad et al., 2021; Gan et al., 2018), which monitor program execution, offer high precision. Yet, their effectiveness relies heavily on the availability of well-designed test cases. These methods do not inherently generate new test inputs to explore various execution paths within concurrent programs. As a result, a large number of test cases is often needed to effectively trigger interleaving across different defect paths, posing a significant challenge in achieving comprehensive coverage.

Fuzzing has emerged as a more efficient method for generating test cases at runtime, effectively exploring a program's execution paths to uncover hidden vulnerabilities and addressing the limitations of other detection techniques (Li et al., 2019b; Wang et al., 2020). Tools like AFL++ (Fioraldi et al., 2020) enhance the discovery of diverse code paths by generating a multitude of test cases, thereby improving code coverage and increasing the likelihood of identifying crashes. However, this approach primarily targets behaviors common to most seed inputs, often overlooking the unique characteristics that individual seed inputs may provide. Meanwhile, traditional fuzz testing tools (Sharma et al., 2012; Forejt et al., 2017; Liu et al., 2021) face limitations in exploring thread interleavings (Rao et al., 2021). These tools rely on sequential code coverage as a feedback mechanism, which often fails to account for the complexities and interactions introduced by thread interleavings in concurrent programs.

2.2. Model checking for concurrency vulnerability detection

Model checking is a formal verification technique that systematically explores a system's state space to validate its correctness, particularly with respect to concurrency properties such as deadlocks, race conditions, and synchronization errors. In concurrent systems, model checking provides a rigorous approach to ensuring that all possible thread or process interleavings lead to valid states, making it an indispensable tool for verifying systems where concurrency is critical. A widely used model checking tool, SPIN (Holzmann, 2005), has been successfully applied to verify the correctness of complex distributed systems. Similarly, UPPAAL (Behrmann et al., 2006) is specifically tailored for verifying real-time and multi-threaded systems. For example, Gu (2020) demonstrated UPPAAL's ability to identify concurrency vulnerabilities in a large-scale autonomous vehicle system model, uncovering race conditions that traditional testing methods failed to detect. Despite its effectiveness, model checking faces significant challenges, with the state space explosion problem being a primary concern. As system complexity increases – especially in systems with numerous threads or intricate synchronization mechanisms – the number of possible states grows exponentially. This exponential growth makes exhaustive state exploration computationally infeasible for large-scale concurrent systems.

2.3. Mutation testing

Mutation testing techniques were first applied to detect concurrency vulnerabilities by King et al. in 1991 (King and Offutt, 1991). Over the subsequent decades, researchers have increasingly integrated mutation testing with concurrency bug detection. Long et al. (2004) proposed the use of mutation monitors to detect concurrency faults, focusing on validating the correctness of replicable components in concurrent programs. Copty and Ur (2007) discussed the Binary Search Algorithm for precise fault localization in concurrent programs. Gligoric et al. (2010) introduced MuTMuT, a mutation testing tool for multi-threaded code that reduces mutation execution time using data-flow fault localization techniques. However, MuTMuT has been criticized for its lack of real-world code testing, making its contributions more theoretical. Wu and Kaiser (2010) proposed an alternative approach using a general mutation testing framework. Experimental evaluations showed that new first-order and second-order SYN central mutation operator sets were more effective and applicable in mutation testing. These studies have shifted the focus toward large-scale foundational software, employing various static analysis methods to optimize the mutation process for improved effectiveness. By leveraging static analysis to refine mutation operators and the execution process, and by using a mutation candidate selection mechanism to minimize the inclusion of ineffective samples in the dynamic testing phase, the aim is to reduce costs while enhancing the scalability and relevance of mutation testing in detecting vulnerabilities during execution.

2.4. Methods involving abstract syntax tree

Abstract syntax trees (ASTs), as representations of program syntax structures, have been widely used in testing technology to locate program defects (Sethi et al., 2024). Pham et al. (2010) utilized ASTs to extract various element features from programs and compared these features with vulnerability descriptions in existing databases to detect vulnerabilities. Li et al. (2017) proposed a method called DP-CNN, which leverages convolutional neural networks. This method uses ASTs to extract token vectors, which are then input into a convolutional neural network to automatically learn the semantic and structural features of the program. Wang et al. (2019) introduced the grey-box fuzzing method Superion, which incorporates a syntax-aware mutation strategy. After performing static analysis of the program's syntax, Superion employs a generative model to learn the syntax, ultimately generating test cases that adhere to the identified rules. Cai et al. (2019a) proposed a software defect prediction method based on the heavy child nodes of ASTs. This method starts from the root node and traverses the AST layer by layer to identify the most significant nodes

at maximum depth in the code, thereby enhancing feature extraction accuracy through the identification of heavy child nodes. These studies demonstrate that converting program source code into ASTs can more precisely pinpoint locations, facilitating operations such as deleting, inserting, and replacing nodes in the code.

In the context of traditional dynamic testing and mutation testing, researchers have employed various static analysis methods to optimize the mutation process and achieve improvements in mutation testing. Integrating static analysis into the mutation phase of dynamic testing provides technical feasibility support for our study. We use static analysis methods to optimize mutation operators and the execution process of mutations. Additionally, we employ mutation filtering mechanisms to minimize the inclusion of ineffective samples in the dynamic testing phase. Ultimately, this approach aims to reduce overhead while maintaining the scalability of mutation testing and the effectiveness in detecting execution-related vulnerabilities.

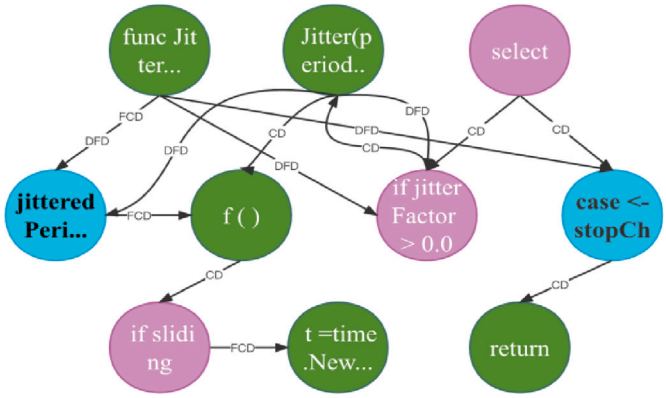

Fig. 2. Example of a code knowledge graph. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3. Approach

This section explores two dimensions of implementing the static fuzz mutation testing based on AST. The first dimension involves vulnerability data and feature mutation operators. It entails analyzing and categorizing vulnerability submissions in large programs built with Go, and utilizing a portion of data and technologies such as knowledge graphs to acquire a set of mutation operators that can be executed at the granularity of abstract syntax trees. The second dimension concerns the implementation of static fuzz mutation method. This involves introducing explaining how the adaptive mutation operator selection algorithm and branch pruning heuristic algorithm we proposed operate and intervene in the entire mutation testing process.

3.1. Vulnerability feature mutation operators

With the rise of vulnerability analysis techniques based on machine learning and neural network models, researchers have extensively studied code representations for vulnerability analysis (Li et al., 2018; Zhou et al., 2019; Cheng et al., 2021). Methods such as data flow graphs (Li et al., 2018), abstract syntax trees (Zhou et al., 2019), and structured program data flows (Cheng et al., 2021) have been employed to model code semantics more accurately. Our goal is to utilize knowledge graphs to assist in identifying vulnerability features within vulnerable code and to develop fine-grained mutation operators at the abstract syntax tree level using feature template-based collections of terms. By analyzing vulnerable code sets from open-source projects, we aim to identify various concurrency vulnerability features present in the code (Zheng et al., 2023). Accurately identifying these vulnerability features is crucial, and knowledge graphs can facilitate this process by modeling features as relationships between entities and attributes. This section first introduces the relevant techniques for constructing a vulnerability code knowledge graph and then discusses the process of deriving mutation operators from the vulnerability code by characterizing these features, which is central to the research presented in this paper.

3.1.1. Construction of vulnerability code knowledge graph

A vulnerability code knowledge graph leverages code analysis tools to abstract elements such as functions, variables, methods, and other components from vulnerable code into nodes, with the relationships between these nodes representing the structure and dependencies within the code. By constructing such a graph and analyzing collections of vulnerable code from open-source projects, various characteristics of concurrency vulnerabilities can be identified. Extracting these vulnerability features is essential for accurate detection, as the knowledge graph models these features through relationships between entities and attributes, facilitating the retrieval of relevant information from the code. Moreover, manual analysis of the vulnerability knowledge graph provides deeper insights into the root causes of each vulnerability and helps pinpoint the specific code statements where these vulnerabilities arise, enabling more targeted and effective remediation.

In this section, we use a vulnerability fix commit (ID: 842f15c3c62a...) from the Kubernetes open-source codebase as an illustrative example.[1] We extract and highlight the core elements of the related context code from this commit, showcasing the knowledge graph generated from it and the valuable insights it provides.

To construct a vulnerability code knowledge graph, we leverage models such as MDERank, CNN, and LSTM-CRF to extract submission information, code modifications across different versions, and relationships from vulnerability submission data. Pre-training is performed using knowledge corpora specifically curated for the software security domain. Entity linking and coreference resolution are employed to cluster identical entities and events effectively. Entity linking addresses entity coreferences by matching them to existing entities within the corpora, while heuristic rules are applied to group unlinked mentions and names in the documents. To detect relationships, entity type constraints and dependency patterns are utilized, enabling the identification and classification of relationships into fine-grained types. This process culminates in the creation of a comprehensive knowledge graph representing vulnerability data.

Fig. 2 provides a simplified visualization of the knowledge graph generated from the code analysis. Neo4j, a widely used graph database, is employed for its efficient retrieval capabilities, powered by the Cypher query language. Cypher is utilized to insert individual nodes into Neo4j, enabling flexible manipulation of code statement nodes and segment-by-segment analysis, as required for this study. In Fig. 2, green nodes represent function entities, purple nodes denote branch entities, and blue nodes correspond to assignment entities. To enhance clarity, some nodes have been omitted, and nodes that combine assignment and declaration within the same statement have not been fully separated.

In Fig. 2, relationships between nodes are represented as follows: if node a is control-dependent on node b, this is denoted as $CD(b, a)$ in the graph. Similarly, if node a is data-dependent on node b, it is expressed as $DFD(b, a)$. Additionally, if function b relies on node a for its invocation, the relationship is represented as $FCD(a, b)$. Each node in the graph corresponds to a specific line of code. To enhance the clarity of relationships between code nodes, Fig. 2 simplifies the visualization by omitting certain non-control nodes. It focuses on highlighting the data and function dependency relationships between code statements, making the graph more intuitive and easier to interpret.

[1] https://github.com/Static-bugs-Detection-Club/Go-SFAST-MT.

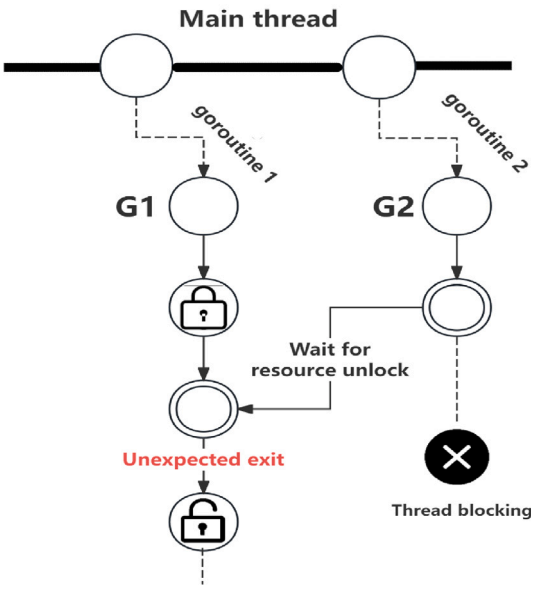

Fig. 3. Mutex deadlock.

3.1.2. Obtaining and analyzing data

To collect a comprehensive set of concurrency vulnerability samples from eight open-source projects, we initially filter the GitHub commit history of these projects. This filtering method involves searching for keywords related to Go language concurrency vulnerabilities, such as "deadlock", "cond", "concurrency", "mutex", "lock", "race", "context", and others. Based on research into Go concurrency vulnerabilities and a preliminary analysis of the initially filtered commit codes, we primarily categorize Go's multi-threaded code into two main types: blocking and non-blocking.

These two types of concurrency vulnerabilities can be further classified according to different triggering mechanisms. Blocking vulnerabilities typically arise when one or more goroutines are executing operations while waiting for resources that remain unavailable. We classify blocking vulnerabilities into two dimensions: those caused by blocking operations due to shared memory access and those caused by message passing operations. The dimension of shared memory access involves using shared variables for inter-thread communication and synchronization. In contrast, message passing operations involve communication and synchronization between processes using channels. In Go, there are five main types of blocking vulnerabilities: mutex deadlocks, race conditions with shared variables, livelocks, channel misuse, and context confusion. Below, we provide a more detailed description of mutex deadlocks:

Mutexes are the most commonly used locks in Go, designed to ensure that only one goroutine can access a shared resource at any given time. However, if a deadlock occurs during their usage, the program may enter an infinite waiting state, effectively halting its execution. Fig. 3 visually illustrates this process.

In Fig. 3, two goroutines, $G1$ and $G2$, both need to access the same resource, A. When $G1$ locks resource A, it successfully enters the critical section associated with A. Meanwhile, $G2$ remains in a waiting state, unable to proceed until A is released. However, if $G1$ subsequently attempts to access another resource and either waits indefinitely or exits the critical section under certain conditions, this prolonged waiting can lead to $G2$ being perpetually blocked while waiting for A. Such a scenario can result in a deadlock, rendering the application unresponsive or prone to errors.

Similar to blocking vulnerabilities, non-blocking vulnerabilities are also prevalent in programs. However, categorizing non-blocking vulnerabilities is less straightforward. The primary types of non-blocking vulnerabilities include race conditions, data races, and misuse of channels. The key distinction between non-blocking and blocking vulnerabilities lies in their effects: blocking vulnerabilities occur when goroutines are halted, preventing the program from continuing execution, whereas non-blocking vulnerabilities stem from issues in the program's logic or data processing, leading to anomalies or crashes.

3.1.3. Generation of vulnerability feature mutation operators

Building on the categorization of vulnerability trigger mechanisms discussed in the previous section, we analyze the root causes of each type of vulnerability by examining the code fixes associated with each category. This analysis enables the generation of a set of characteristic terms that simulate the processes by which vulnerabilities arise. To enhance the representation of embedded vulnerability features and relationships within the knowledge graph, this paper utilizes feature templates to extract terms and assertion sets, which aid in generating mutation operators. The feature templates are specifically designed to capture the characteristics of concurrency vulnerabilities in Go. From these templates, assertions that embody the evolutionary patterns of vulnerabilities (ABOX) are derived. To construct assertion sets for various categories of vulnerabilities, a feature template $G = (T, F)$ is defined, where T and F represent the template's structure and the key locations where vulnerabilities are likely to occur, respectively. The code snippets corresponding to F are used to populate the rule template, representing specific vulnerability features. The populated content forms individual terms. Fig. 4 illustrates the workflow for generating mutation operators using predicate logic and vulnerability features extracted from the knowledge graph.

The feature template acts as a standardized pattern for similar vulnerabilities, forming the foundation for extracting individual terms and predicates. By collecting a substantial number of precise vulnerability features, a comprehensive set of vulnerability terms is established. Subsequently, the mapping relationships between roles and concepts are reorganized to derive assertions that capture the evolutionary patterns of vulnerabilities, resulting in the formation of a new set of assertions. Finally, logical transformations are applied to the term and assertion sets within the feature templates, enabling the generation of mutation operators.

Taking the process of obtaining the RCMX mutation operator as an example, we derived the feature template $G = (T, F)$ from the graph information of multiple code slices, where T represents the relationships between key positions of the vulnerability, including "goroutine", "channel", "select", "mutex", and "waitgroup". F represents the key positions where vulnerabilities occur, including "lock", "unlock", "send", "receive", and "close". Based on a set of code slices, we identify "Method-X" as F to populate the feature template. "Method-X" is then mapped to a role within the relationships between key positions, such as "goroutine". By reorganizing these mapping relationships, we obtain assertions that capture the principles of vulnerability evolution and construct a new set of assertions. For instance, a possible assertion is: "If 'Method-X' is missing in a goroutine, there may be a risk of missing concurrency protection", which corresponds to a fine-grained mutation operator, the RCMX mutation operator. This operator searches the syntax tree of the program for the invocation of Method-X and removes it, generating a new version of the code without Method-X. Similarly, we derived 26 mutation operators related to concurrency vulnerabilities in the Go language, as detailed in Table 2.

In addition to the 26 mutation operators derived from concurrency vulnerability features listed in the table, traditional mutation testing can involve up to 20 additional mutation operators. To apply these operators during static fuzzing mutations on the abstract syntax tree, we use the "go/ast", "go/parser", and "go/token" packages, which are designed to support the Go language's AST. These packages allow us to directly convert source code into a syntax tree and perform operations.

For example, consider the "Modify Sync Fairness" (MSF) operator. The MSF operator begins by checking if a node is a select statement and iterates through each case clause to determine if it includes a channel

Fig. 4. Mutation operator generation flow.

Table 2
26 vulnerability feature mutation operators related to multithreading in Go language obtained in this paper.

Mutation operator name	Mutation operator function
Swap Lock-Unlock Pairs (SLUP)	Swap Lock Operation
Modify Latchand Barrier Thread Count (MLBTC)	Modifying Thread Synchronization Statements
Remove Explicit Critical Section (RECS)	Delete Critical Zone
Modify Latchand Barrier Thread Count (MBRP)	Modifying Barrier Parameters
Shift Explicit Critical Region (SECR)	Moving a Critical Zone
ShrinkCR (SRC)	Shrinking a Critical Zone
Modify Mutex and RWLock Thread Count (MMRTC)	Modify Counter
Remove Concurrency Method-X Call (RCMX)	Removing Concurrent Calls
Remove Concurrency Sync (RCS)	Removing the Synchronization Mechanism
Remove Finally Around Unlock (RFAL)	Removing Unlock Exception Handling
Modify Cond Timedwait Time Value (MCTV)	Modifying the Wait Time
Remove Cond Timedwait Call (RCTC)	Deleting Conditional Variables
Remove Cond Signal Call (RCSC)	Deleting a Signal
Remove Cond Broadcast Call (RCBC)	Deleting Broadcast Calls
Swap Cond Broadcast With Signal (SCS)	Swap Broadcast and Signal Operations
Sync Block Param Swap (SPS)	Swapping Synchronization Block Operations
Modify Sync Fairness (MSF)	Modifying Synchronization Primitive Security
Modify Semaphore Permit Count (MSPC)	Modifying Signal Count
Replace Concurrency Mechanism (RCM)	Replacing the Concurrency Mechanism
Remove Go Statement (RGS)	Remove Go Function Statements
Replace While with If inside Synchronized (RWS)	Replace While and If Statements
Remove Atomic Operation (RAO)	Remove Atomic Operations
Replace Atomic Store with Non-Atomic Store (RASN)	Replacing Atomic Operations With Non Atomic Operations
Swap Lock-Unlock Pairs (SLUP)	Swap Locking and Unlocking Order
Replace Atomic Read-Modify-Write (RA)	Reads and Writes to Cross-Threaded Operations
Modify Method-X Time (MXT)	Modifying the Sleep Time

receive operation. If such an operation is found, the operator replaces it with a custom fair buffer. This buffer ensures that the first thread always continues execution and randomly selects another thread with a 0.5 probability. Specifically, the operator creates a binary expression to compare a random value from the buffer with 0.5 to decide whether to proceed. It then creates a new case clause where the communication operation invokes the Receive method of the custom buffer, while the Body section of the new case clause remains unchanged from the original. Finally, the operator replaces the original case clause with the newly created one. Fig. 5 illustrates a schematic of this mutation operator's execution, featuring a select statement with two case clauses: the first involves a receive operation on channel "ch", and the second on channel "done".

In Fig. 5, the MSF mutation operator modifies channel operations within select statements to introduce greater runtime uncertainty and randomness. The operator replaces the original channel operations with a custom fair buffer. The fair buffer is designed such that the first thread always passes unconditionally, while other threads have a 50% probability of passing. This modification increases the program's randomness, making it more challenging to predict and analyze.

This section provides a brief overview of the implementation logic of a representative mutation operator, highlighting the differences in its execution.

3.2. Static fuzz mutation method

The abstract syntax tree static fuzz mutation technique, serving as the mutator for mutation testing, is the core of the mutation testing system in this study. This section will further investigate a adaptive mutation operator selection algorithm for mutation operator weights based on abstract syntax tree static fuzzing techniques. Additionally, a branch clipping heuristics algorithm for AST will be designed to control the activation or deactivation of the mutation operator to mitigate overfitting. The overall framework of the main algorithm described in this section is illustrated in Fig. 6.

Two algorithms dynamically adjust the mutation process based on the kill rate from mutation testing results. The adaptive mutation operator selection algorithm modifies the probability of selecting mutation operators, while the branch clipping heuristic algorithm controls the number of mutant. The specific implementations of these two algorithms will be introduced below.

3.2.1. Adaptive mutation operator selection algorithm

To effectively apply static fuzzing in practical testing systems, we designed the Mutation Operators Perform Sequence Generation algorithm to address potential overfitting issues. This algorithm decomposes the static fuzzing mutation strategy into multiple components, intervening in the mutation testing system SFAST-MT (Mutation Testing Based on Static Fuzz Mutation of Abstract Syntax Trees) at various stages to optimize the mutation execution process. We specifically focus on the adaptive mutation operator selection component. To integrate seamlessly with the mutation testing system and avoid separate executions solely for weight calculations, the adaptive mutation operator selection algorithm described in this paper distributes multiple function modules within the mutator of the SFAST-MT system. The core part of this algorithm is outlined in Algorithm 1.

The adaptive mutation operator selection algorithm dynamically adjusts the weights of mutation operators based on the kill rate of the mutant during test case execution. This allows for the selection of operators with updated weights for subsequent code testing segments. We will provide a detailed introduction to the three main components involved in Algorithm 1:

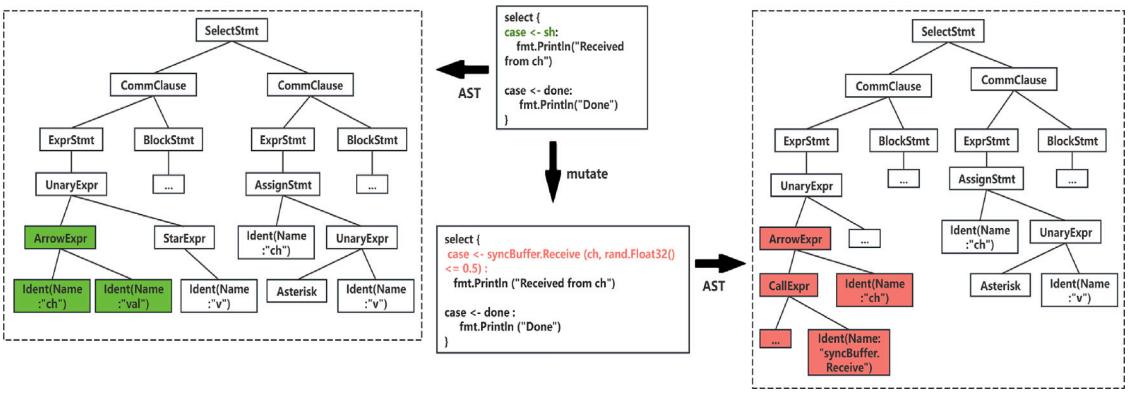

Fig. 5. MSF operator mutation execution.

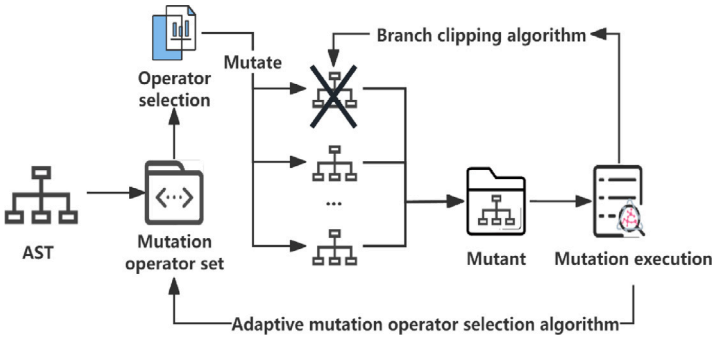

Fig. 6. The overall framework for static fuzz mutation method based on abstract syntax trees, which is also the core component of the green framework section in Fig. 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Algorithm 1: Adaptive Mutation Operator Selection Algorithm

Input: The set of mutant *TotalTestCases*, The set of mutational operators *MutationOperators*, The base killed rate *BaseKilledRate*, The learning rate *LearningRate*
Output: Pending mutation operator *MutationOperator*

```
1  Initialize weights for all operators with InitialWeight
2  foreach operator ∈ MutationOperators do
3      operator.killedCount ← CountKilledTestCases(operator)
4      operator.killedRate ← operator.killedCount / len(TotalTestCases)
5      operator.weight ← operator.weight + LearningRate ×(operator.killedRate -
         BaseKilledRate)
6  end
7  TotalWeight ← SumOfWeights(MutationOperators)
8  Update the weights of each operator through operator.weight / TotalWeight
9  randomValue ← RandomNumberBetween(0, 1)
10 cumulativeWeight ← 0
11 foreach operator ∈ MutationOperators do
12     cumulativeWeight ← cumulativeWeight + operator.weight
13     if randomValue ≤ cumulativeWeight then
14         return MutationOperator ← operator
15     end
16 end
```

Step 1: Initialize Weights (Line 1). The algorithm initializes the weights of the mutation operators. Each operator is assigned a randomly generated initial weight. The initialization of weights is performed only once at the beginning of the mutation testing process. Therefore, the algorithm will accumulate the weights of all operators during subsequent traversal and perform a normalization process **(Lines 7–8)**.

Step 2: Update Weights (Lines 2–6). Here, the algorithm updates the weights of the mutation operators. It adjusts each operator's weight based on the difference between the killed rate during dynamic execution and a base kill rate, using a learning rate to control the magnitude of weight adjustments. In the context of concurrency vulnerabilities, the learning rate is adjusted to ensure better convergence of weights with a limited number of samples.

Step 3: Select Mutation Operator (Lines 9–16). During this step, the mutator generates a random number between 0 and 1 and iterates through the list of mutation operators. It accumulates the weight of the current operator into *cumulativeweight*. If the accumulated weight exceeds or equals the random number, the current mutation operator is selected. The cumulative weight defines a weight interval, allowing operators with larger weights to occupy a larger portion of the interval, thus increasing their probability of being selected.

The adaptive mutation operator selection algorithm enhances the effectiveness of mutation testing by continuously adjusting operator weights during the testing process. The core idea is to modify the weight adjustment amounts based on the kill rates of mutant. This allows the mechanism for updating operator weights to be adapted as the focus of mutation testing shifts. Additionally, the algorithm emphasizes mutant with lower kill rates, as these often indicate defects or potential concurrency vulnerabilities that the test cases might not have detected. By dynamically adjusting operator weights, the mutation testing system can uncover unknown concurrency errors and various edge cases, thereby improving software quality and stability.

3.2.2. *Branch clipping heuristics algorithm*

Relying solely on the self-updating weight algorithm may still result in a substantial number of ineffective samples, particularly higher-order mutations that could diminish the efficiency of mutation testing. To mitigate this issue, a branch pruning heuristic algorithm can be employed. This algorithm prioritizes the mutation of branches with higher mutation potential based on static analysis information from test case execution after mutation. By focusing on more promising branches, branch pruning reduces the occurrence of ineffective mutations, thereby enhancing the overall efficiency of the mutation testing process.

Our proposed static fuzzy mutator does not include a predefined condition for stopping the mutation loop. To address this, a stop condition must be set for each code sample test, except in specific operator mutation testing modes. This section introduces a heuristic algorithm that provides a flexible stopping mechanism, allowing higher-order mutation testing to conclude at an appropriate point, facilitating subsequent manual analysis of the samples. The heuristic algorithm for mutation branch pruning defines two adjustable variables: the threshold for the number of times a mutation operator intervenes and the threshold for the range of mutation score changes, as detailed in algorithm 2.

Algorithm 2: Branch clipping heuristics algorithm

```
   Input: Operator count threshold Threshold, Mutation score MutationRange, Total
          number of operators N
   Output: Mutation termination signal StopMutation, Markers MarkSample
1  foreach mutant ∈ Mutants do
2      operatorVector ← Set N components of vector to 0 ⇒ e_N(0,0,0,...,0)
3      previousKilledRates ← Empty List
4      markSample ← False
5      for i ← 1 to NumberOfMutations do
6          selectedOperator ← SelectOperatorByWeight
7          if Threshold ≤ operatorVector[selectedOperator] then
8              StopMutation
9              break
10         end
11         mutant ← Mutate(mutant, SelectedOperator)
12         result ← RunTestCase(mutant)
13         killedRate ← CalculateKilledRate(result)
14         if |killedRate − Average(preKilledRates)| > MutationRange then
15             StopMutation
16             markSample ← True
17             break
18         end
19         UpdateOperatorVector(operatorVector, SelectedOperator)
20         preKilledRates.Append(killedRate)
21     endfor
22 end
23 Function UpdateOperatorVector(operatorVector, selectedOperator):
24     operatorVector[selectedOperator] ← operatorVector[selectedOperator] + 1
25 end
```

In Algorithm 2, during high-order mutation mode, the algorithm is activated by default. The user must input the threshold values for both the number of interventions by mutation operators and the range of mutation score changes. The mutator utilizes the vector operatorVector to track whether each operator has been executed. Additionally, the mutation score for each round of mutation is stored in a list. Line 7 of Algorithm 2 specifies that if the recorded execution count for a selected operator exceeds the intervention threshold, the mutation process for that sample is immediately terminated. This approach prevents excessive mutation operations, reduces the number of mutant, and avoids unnecessary expansion of the mutation process, thereby enhancing the overall efficiency of mutation testing.

If the first condition is met, mutations and testing proceed as usual. After assessing the kill rate of the mutant, the program evaluates the results at line 14 of Algorithm 2. Each operator execution generates a number of mutant, each with varying kill rates. When the kill rate of a particular sample significantly deviates from its previous mutation testing performance, those samples are flagged for manual inspection. Continuing mutations after observing significant fluctuations in kill rates within a batch of samples is often highly inefficient. To address this, we propose introducing an interface for setting fluctuation thresholds. If the intervention of a particular operator causes excessive fluctuation in the mutation test results of a sample batch, the process is halted, and the occurrence is recorded. This approach aims to penalize mutant that exhibit drastic changes in kill rates, mitigating their impact on subsequent mutation operations.

Overall, the branch clipping heuristic algorithms dynamically adjust the weights of mutation operators, limit the expansion of mutations, and penalize mutation branches. These algorithms complement static fuzzing techniques by offering supplementary benefits. They allow for a selective focus on mutation operators and reduce further mutations of unstable samples, thereby increasing the likelihood of identifying concurrency vulnerabilities within the samples.

3.3. Mutation testing system: SFAST-MT

The static fuzz mutation approach based on abstract syntax trees, which we have designed, will function as the mutator for mutation testing tools. We have implemented a mutation testing system named SFAST-MT (Mutation Testing Based on Static Fuzz Mutation of Abstract Syntax Trees). This system performs mutation testing at the syntax tree level using mutation operators enriched with concurrency vulnerability features.

The execution of mutation testing in SFAST-MT occurs in four main stages: setting the source code paths for mutation, configuring disabled operators and the modes of the static fuzzy mutator, cleaning up temporary files, and reporting mutation scores for each sample. Each stage involves calling functions from the corresponding files independently and in a stateless manner. During runtime, the system collects all non-test Go files from the target directory, transforms them into abstract syntax trees according to the mutator's settings, and applies the mutations.

This section provides a straightforward example of code mutation testing to illustrate the operational process of the SFAST-MT system. Fig. 7 shows the source code samples used in mutation testing and the corresponding test case assertion code. In this case, the adaptive mutation operator selection algorithm will not be employed. Instead, only the following four operators are selected: "Modify Expression Operands", "Remove Assignment Statements", "Remove If Regions", and "Remove Else Regions".

Based on the source code and mutation testing examples provided in Fig. 7, the objective is to determine whether the output can distinguish between mutations applied by four mutation operators on the code. These operators target the deletion of if-condition branch statements, deletion of else-condition branch statements, replacement of expression operators, and deletion of regular statements (non-branching statements, etc.). Theoretically, this should generate 6 samples for the *baz()* function: 2 samples for the deletion of if and else statements, 1 for operator replacement, and 3 for statement deletions. However, the SFAST-MT system automatically detects and removes duplicate temporary mutant, resulting in only 5 mutant being processed during execution. Since mutants are generally quite lengthy, this system only displays those mutants that have not been killed. And to simplify the output, only the code fragments corresponding to the mutated sections are shown.

4. Experiments

In this section, we focus on evaluating concurrency vulnerability feature mutation operators and the mutator incorporating adaptive mutation operator selection and heuristic algorithms. Our experimental validation is twofold. First, we demonstrate that vulnerability feature operators are more effective in generating samples with concurrency vulnerabilities compared to traditional mutation operators, using code from actual project vulnerability commit fixes. Second, we compare traditional mutation testing with a random mutator against a mutator equipped with adaptive mutation operator selection and heuristic algorithms. By examining the impact of these algorithms on metrics such as equivalent samples and mutation scores, we validate their effectiveness in enhancing the simulation of vulnerability features.

4.1. Experimental data and setup

Firstly, we need to validate that vulnerability feature operators are more effective at generating samples with concurrency vulnerabilities

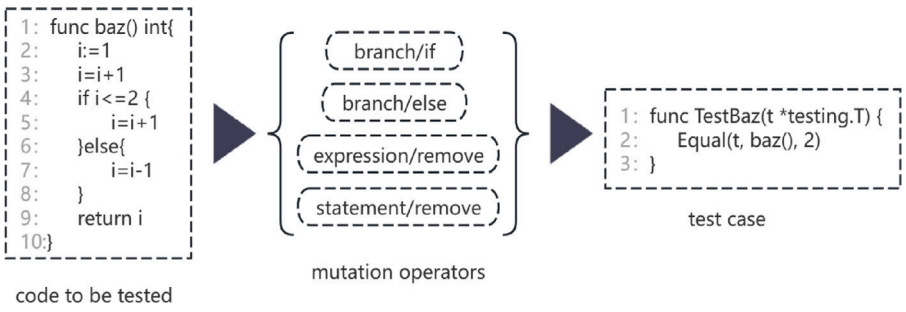

Fig. 7. Example of the SFAST-MT test process.

compared to traditional mutation operators. To ensure the accuracy of this experiment, all mutation tests are restricted to a maximum of second-order mutations. We do not optimize the weights of mutation operators or use additional heuristic pruning algorithms to terminate the mutation process. The choice to avoid higher-order mutations is to focus on examining the impact of different mutation operator sets on the mutant, as variations in operator sets would influence weight calculations. To prevent overfitting, this study conducts a comparative experiment between 20 traditional mutation operators and 26 concurrency vulnerability feature operators. The experimental data comprise 173 code slices from eight large open-source projects, with 73 submissions related to blocking and 81 related to non-blocking vulnerabilities. The operator set is divided into two groups: the first group includes only the 20 traditional operators, while the second group combines these 20 traditional operators with the 26 vulnerability feature operators, totaling 46 operators.

Secondly, we need to validate the advantages of mutators equipped with adaptive mutation operator selection and heuristic algorithms compared to traditional mutation testing. To account for the potential impact of different source codes on the results, we used not only code from the eight open-source projects but also selected packages from the Go language standard library. In the golang/go repository on GitHub, Go language packages include files with the .go suffix and corresponding _test.go files, which contain test code related to functions, methods, or types within the package and are used for unit testing assertions. We utilized these test codes, which are likely free of vulnerabilities, to perform comparative tests between different mutators, aiming to minimize any bias introduced by the interaction between the dataset and operators. The source code we used is divided into two parts. For the Go language standard library code, we selected files with a .go suffix that have corresponding _test.go files. For the code from the eight open-source projects, we selected code slices longer than 15 lines, as detailed in Table 3.

Table 3 collected approximately 120 source code slices or code files from two different sources of code repositories, with each source contributing 60. The code files from the Go language standard library are not directly related to concurrency vulnerabilities, and conducting mutation tests on them can reduce the influence of concurrency-related operators in SFAST-MT on the algorithm optimization results. The open-source project commits, are related to concurrency vulnerability submissions, which are the same dataset used in the previous experiments. Therefore, on the code dataset from the open-source project submissions, we will use 20 regular operators along with 26 vulnerability feature operators. In contrast, for the code dataset from the Go language standard library, all vulnerability feature operators of the mutation testing tools will be blocked.

Table 3
Sources of experimental data for comparing static fuzzing algorithms.

Code source	Specific sources	Quantities
Standard library of Go language	go/src/bufio	3
	go/src/bytes	3
	go/src/cmd	17
	go/src/compress	10
	go/src/container	3
	go/src/crypto	24
Open source project	docker	34
Submission	kubernetes	26

4.2. Evaluation measures

When evaluating the effectiveness of software testing techniques, it is essential to consider multiple performance metrics. In the field of mutation testing, common metrics include mutation score, the number of equivalent mutants, and fault detection capability. These metrics are widely used to assess the performance of mutation testing across various software systems and have been proven effective in numerous experimental studies.

- The mutation score (MS) is defined as the ratio of the number of mutants detected during the mutation testing process to the total number of generated mutants. The mutation score is calculated as the ratio of the number of killed mutants (NK) to the number of generated mutants (NG). A killed mutant refers to a mutant that produces a different output from the original program. The higher the mutation score, the more effective the mutation testing is considered to be. It can be represented by the formula (1):

$$Mutation\ Score(MS) = NK\ /\ NG \tag{1}$$

- The equivalent mutant score (EMS) accounts for the test effectiveness excluding the set of equivalent mutants. Equivalent mutants are those that mutation testing does not aim to kill, as they do not help in uncovering faults in the original program. The Equivalent Mutant Score measures how effectively test cases kill non-equivalent mutants. It calculates this by dividing the number of non-equivalent mutants killed (NKNM) by the total number of non-equivalent mutants, which includes both killed non-equivalent mutants and equivalent mutants (NE). It can be represented by formula (2):

$$Equivalent\ Mutant\ Score(EMS) = NKNM\ /\ (NG - NE) \tag{2}$$

Since automatically detecting fully equivalent mutants is challenging in software testing, we manually inspect all generated mutant to identify equivalent ones. The Equivalent Mutant Score (EMS) ranges from 0 to 1, with values closer to 1 indicating better test coverage and mutant killing efficiency. Compared to the mutation score, the EMS is more stringent as it considers only mutants that were not killed, rather than the total number of generated mutants. This metric emphasizes quality over quantity. Many studies have not strictly differentiated between these two methods of calculating mutation scores, as both can accurately reflect the performance of mutation testing (Budd and Angluin, 1982; Zheng et al., 2020). Moreover, using both mutation scores together helps testers evaluate whether the scale of mutants is appropriate.

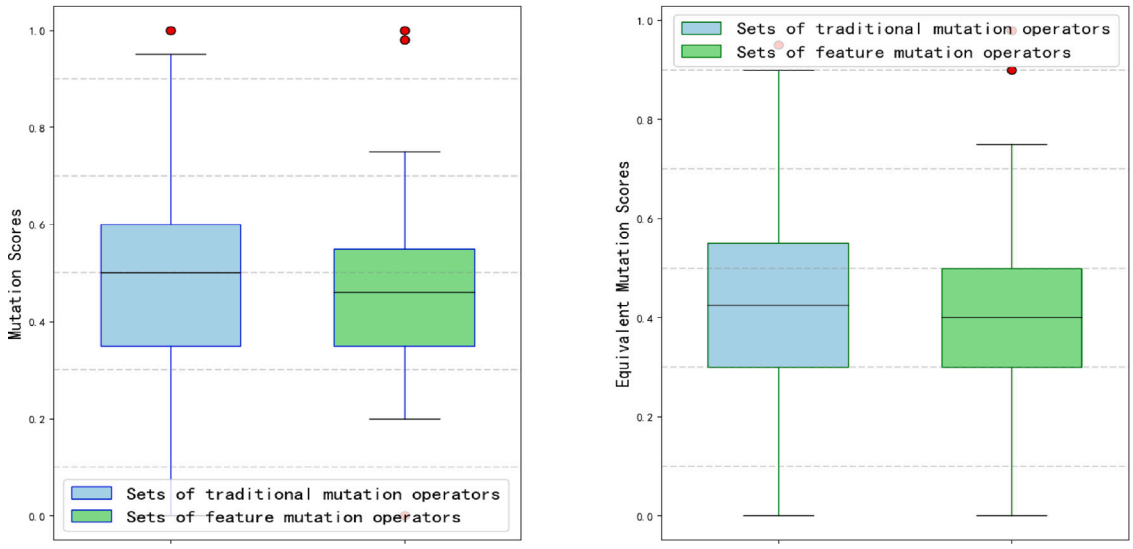

Fig. 8. Mutation scores and equivalent mutation scores for blocking vulnerabilities.

4.3. Experimental results and analysis

To evaluate the effectiveness of a static fuzzing mutation testing system based on Abstract Syntax Trees, we will analyze the experimental results based on the two research questions we previously designed.

4.3.1. RQ1: Are vulnerability feature operators more effective in generating samples with concurrency vulnerabilities compared to traditional mutation operators?

We will conduct a comparative analysis between the vulnerability feature operators proposed in this paper and traditional mutation operators to evaluate their effectiveness in detecting concurrency vulnerabilities. Using the same set of test cases, we will perform mutation testing and record the mutants that were not killed. Subsequently, we will manually analyze these un-killed mutants to assess the impact of the proposed operators on improving the detection of concurrency vulnerabilities.

In this section, we categorize the set of mutation operators into two groups: The first group includes a combination of both the vulnerability feature mutation operators proposed here and traditional mutation operators, while the second group consists solely of traditional mutation operators. We apply the same set of samples for mutation testing to both groups—one group with traditional operators and the other with the proposed specificity operators. The mutation testing system tracks the execution of mutation operators for each sample and notes whether they were killed by the predetermined set of test cases. For mutants not killed by these test cases, we count their occurrences and conduct a manual analysis to determine if they exhibit concurrency vulnerabilities related to the submission. The code from eight open-source projects used in this experiment is divided into blocking and non-blocking sections, with experiments conducted sequentially on each. Non-equivalent and equivalent mutation scores for blocking vulnerabilities are shown in Fig. 8, while the mutation testing comparison results for non-blocking vulnerabilities are presented in Fig. 9.

From Fig. 8, it is evident that for the code dataset associated with blocking vulnerabilities, both mutation scores and equivalent mutation scores exhibit a decline in the group utilizing vulnerability feature mutation operators. Specifically, incorporating these operators resulted in an average reduction of 8.2% in the median mutation score and an 8% decrease in the median equivalent mutation score. The reduction in equivalent mutation scores is attributed to the removal of identical samples during the initial screening process. This declining trend is consistently observed regardless of whether vulnerability feature mutation operators are included. Notably, the median equivalent mutation score decreased by 9.3% following the inclusion of feature operators.

Fig. 9 shows a similar trend in mutation testing for commits related to non-blocking vulnerabilities. After incorporating the vulnerability feature mutation operator set, the median mutation score decreased by approximately 6.3%, while the median equivalent mutation score dropped by about 10%. This indicates that, under the same test case set, the reduction in both scores suggests an increase in vulnerability-related mutants that remain undetected.

These results demonstrate that vulnerability feature mutation operators contribute to generating more mutant that remain undetected by test cases, highlighting their positive impact on enhancing mutation testing effectiveness.

To further investigate whether there is a direct correlation between the decrease in mutation scores and concurrency vulnerabilities, we will conduct a manual analysis of the non-equivalent mutant that were not killed by the test cases. Specifically, the SFAST-MT system records the temporary files of mutant that were not killed by the test cases. We then compare these temporary files with the corresponding bug-fixing commits from GitHub. The results are presented in Fig. 10, which shows the proportion of mutant related to concurrency vulnerabilities among all non-killed mutant.

The proportion shown in Fig. 10 clearly indicates that the experimental group, which incorporates the vulnerability feature mutation operators, contains more commit-related mutant than the control group, which uses traditional operators. Specifically, for non-blocking vulnerability samples, the inclusion of feature operators increased the number of commit-related mutant by approximately 17.3%, while for blocking vulnerability samples, this increase was about 27%. The commit-related mutant directly reflect the improvement brought about by the vulnerability feature mutation operators in fitting the mutation testing framework to detect Go concurrency vulnerabilities. Combining the results from the previous experiments with Figs. 8 and 9, we can conclude that the vulnerability feature mutation operators significantly enhance the mutation testing effectiveness for detecting concurrency vulnerabilities.

4.3.2. RQ2: Do adaptive mutation operator selection and heuristic algorithms offer superior performance compared to traditional random mutators?

We aim to conduct mutation testing on code from different sources with the adaptive mutation operator selection algorithm both enabled and disabled to evaluate its impact on the mutation testing tool. To

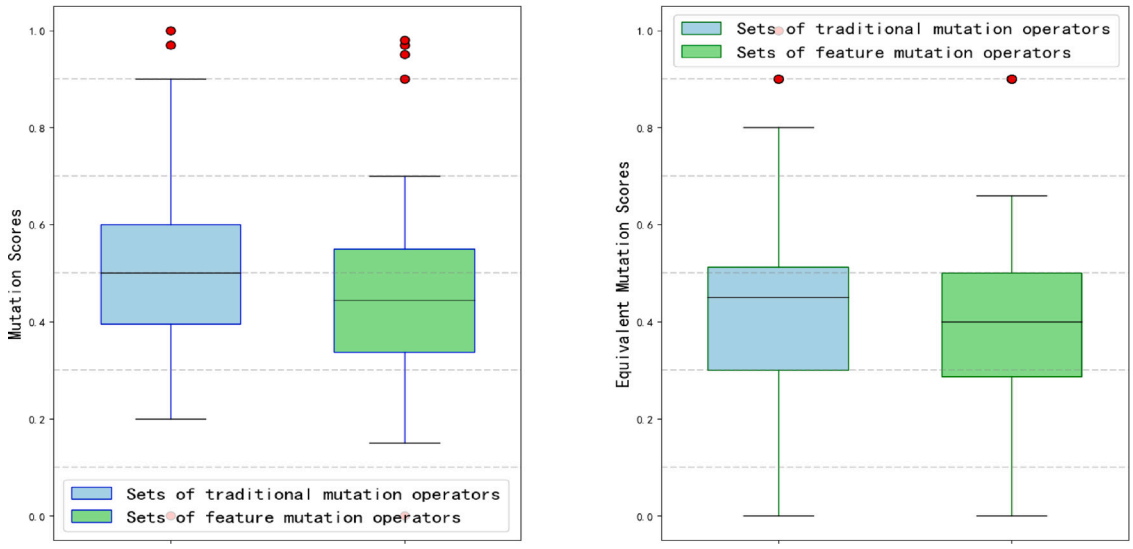

Fig. 9. Mutation scores and equivalent mutation scores for non-blocking vulnerabilities.

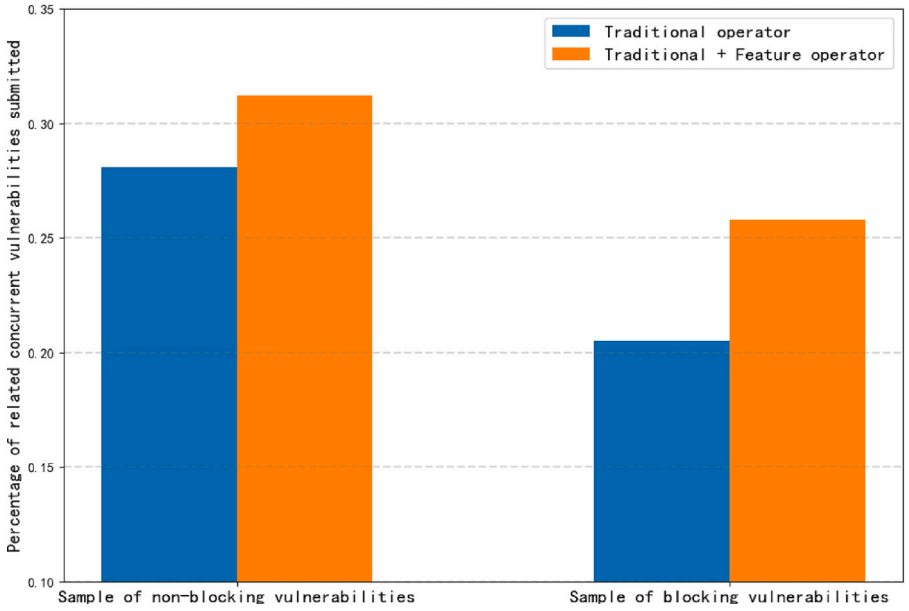

Fig. 10. Percentage of submission-related concurrency vulnerabilities.

provide a comprehensive comparison, we introduce a modified version of the open-source tool Golang-Mutation-Testing[2] as a control group alongside SFAST-MT. The control group, utilizing Golang-Mutation-Testing, performs first-order mutations with 20 traditional mutation operators individually. This control group is introduced to benchmark the performance of other mutation testing systems, with its basic functionalities serving as a standard for comparison. Since the Golang-Mutation-Testing system does not include mechanisms for identifying equivalent or duplicate samples, its equivalent and non-equivalent scores in this experiment are represented by constant test case pass rates within its testing logic. The mutation scores obtained from testing source code from two different sources using three different mutation methods are shown in Fig. 11.

Fig. 11 illustrates the mutation score results for three different mutators across two types of datasets. The mutation score trends for both datasets are nearly identical. The results show that the mutation testing scores for all three mutators on the Go language standard library dataset are higher than those for the project submission source code group. Specifically, the adaptive mutation operator selection algorithm, when compared to purely random operator execution, led to a decrease in the median mutation score by approximately 4.3% for the Go language standard library dataset. In contrast, compared to Golang-Mutation-Testing, the median mutation score dropped by over 20% for both datasets. As depicted in Fig. 11, the final mutation scores for the three methods using the Go language standard library dataset were slightly higher than those for the project submission source code group. This discrepancy is attributed to the more complete and comprehensive nature of the code files and test cases in the Go language standard library. Conversely, the project submission source code group, with its experimental control setup and partial truncation of code slices, exhibited lower test case coverage. Consequently, the mutation scores for the Go language standard library dataset are generally higher in this experiment.

[2] https://github.com/StefanSchroeder/Golang-Mutation-testing.

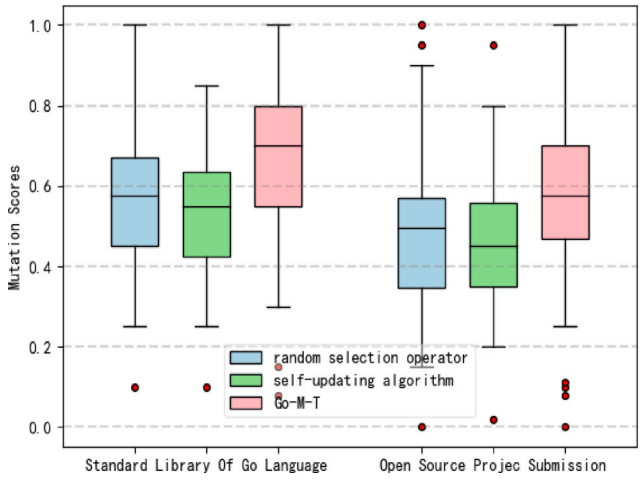

Fig. 11. Different mutation methods and corresponding mutation scores in the source code.

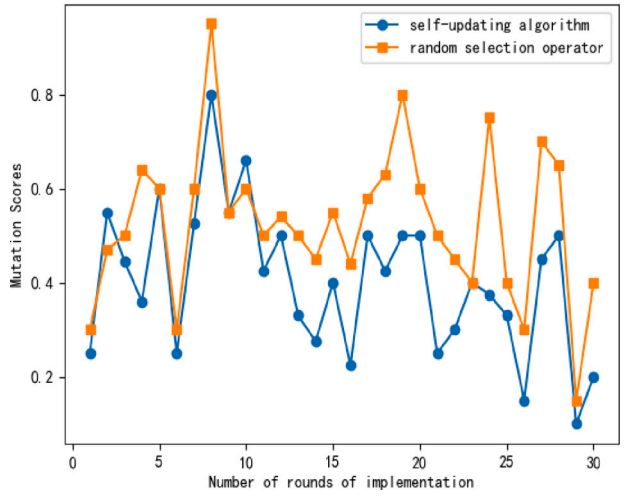

Fig. 12. Code mutation testing results in chronological order.

To illustrate the impact of the adaptive mutation operator selection algorithm on mutation testing more intuitively, we present the results in chronological order, as shown in Fig. 12. We shuffled 60 pieces of source code from two sources and recorded their mutation scores in pairs, according to the order of execution. These scores were then compared with those obtained using the adaptive mutation operator selection algorithm and the Golang-Mutation-Testing system as benchmarks.

Although Fig. 12 displays results for only half of the samples, a noticeable trend emerges: as the number of mutation testing iterations increases, mutation scores with the adaptive mutation operator selection algorithm tend to decrease, and the gap between these scores and those of the benchmarks widens. This trend arises because the self-updating algorithm adjusts operator weights based on the mutation results of previous code segments, thereby influencing the mutation process of subsequent segments.

It is important to note that the experimental sample order is random, and there is significant variance in mutation scores across different samples, influenced by the specific characteristics of the samples and their test cases. Therefore, the curve in Fig. 12 may not clearly reflect the changes in differences.

Overall, the analysis of the mutation testing experimental data indicates that the mutation testing system with the adaptive mutation operator selection algorithm consistently yielded lower mutation scores compared to both fully random mutators and traditional mutation testing systems across both datasets. This suggests that the adaptive mutation operator selection algorithm is effective in generating more mutants that potentially contain vulnerabilities, which might be missed by conventional mutators.

5. Limitations and threats to validity

External validity: The threat to external validity is related to the generalizability of our research findings. This study focuses on concurrency vulnerability features in Go code and potentially harmful vulnerability code. However, different programming languages or types of vulnerabilities exhibit distinct vulnerability features. Therefore, our findings may not be quickly generalized to other programming languages. For example, compared to C/C++, Go language[3] is more prone to triggering concurrency vulnerabilities in our study. Consequently, we are also striving to advance the application of static fuzz mutation to other programming languages and types of vulnerabilities.

Internal validity: The threat to internal validity is related to our use of exploit code for testing static fuzz mutation. We analyzed contributor commit records from 8 projects, including code segments with interfaces such as reads and writes across multiple functions. The efficiency of manually screening large volumes of cross-functional and cross-version code was affected. To minimize the risk of overlooking or misjudging relevant code segments, the entire process was conducted collaboratively by two authors. They engaged in multiple rounds of analysis on ambiguous code until consensus was reached. Furthermore, as we plan to expand this study to include different programming languages in the future, we will continue to augment our dataset of vulnerability code. This expansion aims to address the challenge of limited available data for practical testing purposes.

Construct validity: The threat to construct validity is tied to both the validation method and the approach for optimizing the mutation operator sequence. Although we implemented Algorithm 2 to refine the mutation operator set, the limited size of real vulnerability code slices impedes our ability to definitively assess overfitting. This limitation arises because the static fuzz mutation technique depends on subsequent collaborative filtering and other mutant screening processes within its framework. Therefore, the effectiveness of the static fuzz mutation technique in self-updating mutation operator sequences will be more accurately evaluated as improvements are made to these subsequent technologies.

6. Conclusion and future work

Our research centers on static fuzzing mutation testing based on Abstract Syntax Trees, leveraging concurrency vulnerability feature mutation operators alongside adaptive mutation operator selection and heuristic algorithms for controlling higher-order mutations. The ultimate aim is to enhance the generation ratio of concurrency vulnerability mutants in mutation testing. By analyzing various testing techniques, we seek to optimize mutation testing as a vulnerability detection method. Our comparative experiments utilized data from diverse sources to evaluate the efficacy of feature operators versus traditional static fuzzing algorithms. Results indicate that feature operators consistently reduced mutation scores compared to the control group using only traditional operators. Additionally, the inclusion of feature operators significantly increased the quantity of code relevant

[3] https://go.dev/talks/2013/advconc.slide.

to mutant submissions.

Currently, the performance of our mutation testing approach is limited by the small sample size and the comprehensiveness of the corresponding test suites, making it challenging to fit vulnerabilities with a wide range. To address this, future work will focus on expanding the sample size and test suites to cover a broader array of scenarios and boundary conditions. We plan to incorporate more diverse test cases from real production applications or open-source projects and potentially integrate our mutation testing system with actual vulnerability databases. This will help the system learn patterns from known vulnerabilities and enhance its ability to detect real-world vulnerabilities.

CRediT authorship contribution statement

Wei Zheng: Supervision, Conceptualization. **Chang Liu:** Writing – original draft, Validation, Methodology, Investigation. **Peiran Deng:** Software, Formal analysis, Data curation. **Xiang Chen:** Writing – review & editing, Resources, Conceptualization. **Xiaoxue Wu:** Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that we do not have any commercial or associative interest that represents a conflict of interest in connection with the work. The authors declare the work described was original research that has not been published previously, and not under consideration for publication elsewhere, in whole or in part. All the authors listed have approved the manuscript.

Acknowledgments

This research is supported by the National Natural Science Foundation of China special project capability-based construction method and execution mechanisms for ubiquitous operating systems (62141208) and the National Natural Science Foundation of China (62202414).

Data availability

Data will be made available on request.

References

Ahmad, Adil, Lee, Sangho, Fonseca, Pedro, Lee, Byoungyoung, 2021. Kard: Lightweight data race detection with per-thread memory protection. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 647–660.

Barbar, Mohamad, Sui, Yulei, 2021. Compacting points-to sets through object clustering. Proc. ACM Program. Lang. 5 (OOPSLA), 1–27.

Behrmann, Gerd, David, Alexandre, Larsen, Kim Guldstrand, Håkansson, John, Pettersson, Paul, Yi, Wang, Hendriks, Martijn, 2006. Uppaal 4.0.

Budd, Timothy A., Angluin, Dana, 1982. Two notions of correctness and their relation to testing. Acta Inform. 18 (1), 31–45.

Cai, Ziyi, Lu, Lu, Qiu, Shaojian, 2019a. An abstract syntax tree encoding method for cross-project defect prediction. IEEE Access 7, 170844–170853.

Cai, Yan, Zhu, Biyun, Meng, Ruijie, Yun, Hao, He, Liang, Su, Purui, Liang, Bin, 2019b. Detecting concurrency memory corruption vulnerabilities. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 706–717.

Cao, Sicong, He, Biao, Sun, Xiaobing, Ouyang, Yu, Zhang, Chao, Wu, Xiaoxue, Su, Ting, Bo, Lili, Li, Bin, Ma, Chuanlei, Li, Jiajia, Wei, Tao, 2023a. Oddfuzz: Discovering java deserialization vulnerabilities via structure-aware directed greybox fuzzing. In: 2023 IEEE Symposium on Security and Privacy. SP, pp. 2726–2743.

Cao, Sicong, Sun, Xiaobing, Bo, Lili, Wu, Rongxin, Li, Bin, Wu, Xiaoxue, Tao, Chuanqi, Zhang, Tao, Liu, Wei, 2023b. Learning to detect memory-related vulnerabilities. ACM Trans. Softw. Eng. Methodol. 33 (2), 1–35.

Chen, Dongjie, Jiang, Yanyan, Xu, Chang, Ma, Xiaoxing, Lu, Jian, 2018. Testing multithreaded programs via thread speed control. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 15–25.

Cheng, Xiao, Wang, Haoyu, Hua, Jiayi, Xu, Guoai, Sui, Yulei, 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (3), 1–33.

Copty, Shady, Ur, Shmuel, 2007. Toward automatic concurrent debugging via minimal program mutant generation with AspectJ. Electron. Notes Theor. Comput. Sci. 174 (9), 151–165.

Dam, Hoa Khanh, Pham, Trang, Ng, Shien Wee, Tran, Truyen, Grundy, John, Ghose, Aditya, Kim, Taeksu, Kim, Chul-Joo, 2018. A deep tree-based model for software defect prediction. arXiv preprint arXiv:1802.00921.

Fioraldi, Andrea, Maier, Dominik, Eißfeldt, Heiko, Heuse, Marc, 2020. {AFL++}: Combining incremental steps of fuzzing research. In: 14th USENIX Workshop on Offensive Technologies. WOOT 20.

Forejt, Vojtăch, Joshi, Saurabh, Kroening, Daniel, Narayanaswamy, Ganesh, Sharma, Subodh, 2017. Precise predictive analysis for discovering communication deadlocks in MPI programs. ACM Trans. Program. Lang. Syst. (TOPLAS) 39 (4), 1–27.

Gan, Shuitao, Zhang, Chao, Qin, Xiaojun, Tu, Xuwen, Li, Kang, Pei, Zhongyu, Chen, Zuoning, 2018. Collafl: Path sensitive fuzzing. In: 2018 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 679–696.

Gligoric, Milos, Jagannath, Vilas, Marinov, Darko, 2010. MuTMuT: Efficient exploration for mutation testing of multithreaded code. In: 2010 Third International Conference on Software Testing, Verification and Validation. IEEE, pp. 55–64.

Godefroid, Patrice, 2007. Random testing for security: blackbox vs. whitebox fuzzing. In: Proceedings of the 2nd International Workshop on Random Testing: Co-Located with the 22nd IEEE/ACM International Conference on Automated Software Engineering. ASE 2007, 1–1.

Gosain, Anjana, Sharma, Ganga, 2015. Static analysis: A survey of techniques and tools. In: Intelligent Computing and Applications: Proceedings of the International Conference on ICA, 22-24 December 2014. Springer, pp. 581–591.

Gu, Rong, 2020. Automatic model generation and scalable verification for autonomous vehicles.

Holzmann, Gerard J., 2005. Software model checking with SPIN. Adv. Comput. 65, 77–108.

Inverso, Omar, Nguyen, Truc L, Fischer, Bernd, La Torre, Salvatore, Parlato, Gennaro, 2015. Lazy-cseq: A context-bounded model checking tool for multi-threaded c-programs. In: 2015 30th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 807–812.

Jia, Yue, Harman, Mark, 2010. An analysis and survey of the development of mutation testing. IEEE Trans. Softw. Eng. 37 (5), 649–678.

Jian, Liu, Purui, Su, Min, Yang, Liang, He, Yuan, Zhang, Xueyang, Zhu, Huimin, Lin, 2018. A survey of software and network security. J. Softw. 29 (01), 42–68.

King, Kim N., Offutt, A. Jefferson, 1991. A fortran language system for mutation-based software testing. Softw. - Pract. Exp. 21 (7), 685–718.

Kroening, Daniel, Poetzl, Daniel, Schrammel, Peter, Wachter, Björn, 2016. Sound static deadlock analysis for C/Pthreads. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. pp. 379–390.

Lemieux, Caroline, Sen, Koushik, 2018. Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. pp. 475–485.

Li, Jian, He, Pinjia, Zhu, Jieming, Lyu, Michael R., 2017. Software defect prediction via convolutional neural network. In: 2017 IEEE International Conference on Software Quality, Reliability and Security. QRS, IEEE, pp. 318–328.

Li, Guangpu, Lu, Shan, Musuvathi, Madanlal, Nath, Suman, Padhye, Rohan, 2019a. Efficient scalable thread-safety-violation detection: finding thousands of concurrency bugs during testing. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. pp. 162–180.

Li, Yuekang, Xue, Yinxing, Chen, Hongxu, Wu, Xiuheng, Zhang, Cen, Xie, Xiaofei, Wang, Haijun, Liu, Yang, 2019b. Cerebro: context-aware adaptive fuzzing for effective vulnerability detection. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 533–544.

Li, Zhen, Zou, Deqing, Xu, Shouhuai, Ou, Xinyu, Jin, Hai, Wang, Sujuan, Deng, Zhijun, Zhong, Yuyi, 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681.

Liu, Ziheng, Zhu, Shuofei, Qin, Boqin, Chen, Hao, Song, Linhai, 2021. Automatically detecting and fixing concurrency bugs in go software systems. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 616–629.

Long, Brad, Duke, Roger, Goldson, Doug, Strooper, Paul, Wildman, Luke, 2004. Mutation-based exploration of a method for verifying concurrent Java components. In: 18th International Parallel and Distributed Processing Symposium, 2004. Proceedings. IEEE, p. 265.

Papadakis, Mike, Kintis, Marinos, Zhang, Jie, Jia, Yue, Le Traon, Yves, Harman, Mark, 2019. Mutation testing advances: an analysis and survey. In: Advances in Computers. Vol. 112, Elsevier, pp. 275–378.

Parihar, Mithelesh, Bharti, Anu, 2019. Role of software testing life cycle (STLC) in software development life cycle (SDLC). Int. J. Res. 6 (8), 649–661.

Park, Jihyeok, Lee, Hongki, Ryu, Sukyoung, 2021. A survey of parametric static analysis. ACM Comput. Surv. 54 (7), 1–37.

Pham, Nam H, Nguyen, Tung Thanh, Nguyen, Hoan Anh, Nguyen, Tien N, 2010. Detection of recurring software vulnerabilities. In: Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering. pp. 447–456.

Rao, Lei, Liu, Shaoying, Liu, Ai, 2021. Testing program segments to detect software faults during programming. Int. J. Perform. Eng. 17 (11), http://dx.doi.org/10.23940/ijpe.21.11.p1.907917.

Ray, Baishakhi, Posnett, Daryl, Filkov, Vladimir, Devanbu, Premkumar, 2014. A large scale study of programming languages and code quality in github. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 155–165.

Ren, Zezhong, Han, Zheng, Zhang, Jiayuan, Wang, Wenjie, Feng, Tao, Wang, He, Zhang, Yuqing, 2021. Review of fuzzy testing techniques. Comput. Res. Dev. 58 (5), 944–9638.

Sethi, Anushka, Sangalay, Aseem, Malhotra, Ruchika, 2024. Software defect prediction using abstract syntax trees features and object—Oriented metrics. In: Reliability Engineering for Industrial Processes: An Analytics Perspective. Springer, pp. 189–201.

Sharma, Subodh, Gopalakrishnan, Ganesh, Bronevetsky, Greg, 2012. A sound reduction of persistent-sets for deadlock detection in MPI applications. In: Brazilian Symposium on Formal Methods. Springer, pp. 194–209.

Shastry, Bhargava, Leutner, Markus, Fiebig, Tobias, Thimmaraju, Kashyap, Yamaguchi, Fabian, Rieck, Konrad, Schmid, Stefan, Seifert, Jean-Pierre, Feldmann, Anja, 2017. Static program analysis as a fuzzing aid. In: Research in Attacks, Intrusions, and Defenses: 20th International Symposium, RAID 2017, Atlanta, GA, USA, September 18–20, 2017, Proceedings. Springer, pp. 26–47.

Tu, Tengfei, Liu, Xiaoyu, Song, Linhai, Zhang, Yiying, 2019. Understanding real-world concurrency bugs in go. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 865–878.

Wang, Junjie, Chen, Bihuan, Wei, Lei, Liu, Yang, 2019. Superion: Grammar-aware greybox fuzzing. In: 2019 IEEE/ACM 41st International Conference on Software Engineering. ICSE, IEEE, pp. 724–735.

Wang, Haijun, Xie, Xiaofei, Li, Yi, Wen, Cheng, Li, Yuekang, Liu, Yang, Qin, Shengchao, Chen, Hongxu, Sui, Yulei, 2020. Typestate-guided fuzzer for discovering use-after-free vulnerabilities. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. pp. 999–1010.

Wong, W. Eric, 2001. Mutation Testing for the New Century, vol. 24, Springer Science & Business Media.

Wu, Leon Li, Kaiser, Gail E., 2010. Empirical study of concurrency mutation operators for java.

Yang, Xinyue, Zhang, Xiaofang, Tong, Yao, 2022. Simplified abstract syntax tree based semantic features learning for software change prediction. J. Softw.: Evol. Process. 34 (4), e2445.

Zhang, Jie, Wang, Ziyi, Zhang, Lingming, Hao, Dan, Zang, Lei, Cheng, Shiyang, Zhang, Lu, 2016. Predictive mutation testing. In: Proceedings of the 25th International Symposium on Software Testing and Analysis. pp. 342–353.

Zheng, Wei, Deng, Peiran, Gui, Kui, Wu, Xiaoxue, 2023. An Abstract Syntax Tree based static fuzzing mutation for vulnerability evolution analysis. Inf. Softw. Technol. 158, 107194.

Zheng, Wei, Gao, Jialiang, Wu, Xiaoxue, Liu, Fengyu, Xun, Yuxing, Liu, Guoliang, Chen, Xiang, 2020. The impact factors on the performance of machine learning-based vulnerability detection: A comparative study. J. Syst. Softw. 168, 110659.

Zheng, Wei, Shen, Tianren, Chen, Xiang, Deng, Peiran, 2022. Interpretability application of the just-in-time software defect prediction model. J. Syst. Softw. 188, 111245.

Zhou, Yaqin, Liu, Shangqing, Siow, Jingkai, Du, Xiaoning, Liu, Yang, 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Wei Zheng is currently an associate professor with the Department of Software, University of Northwestern Polytechnical. He received the M.Sc degree in Computer Science Ph.D degree in Computer Software and Theory from University of Northwestern Polytechnical, China. His research focus is on software security software quality assurance. He is the author or coauthor of more than 40 publications of international journals of high impact or prestigious international conferences. Moreover, he has been a reviewer of several international journals and conferences.

Chang Liu is currently a student of the Department of Software, University of Northwestern Polytechnical, for the Master degree. His research focus is on software security.

Peiran Deng is currently a student of the Department of Software, University of Northwestern Polytechnical, for the Master degree. His research focus is on software testing.

Xiang Chen is currently an associate professor with the Department of Computer Science and Technology, Nan Tong University. He received the Ph.D degree in Computer Science and Technology from University of Nan Jing, China. His research focus is on Software defect prediction and machine learning. He is the author or coauthor of more than 60 research publications, which includes international journals of high impact or prestigious international conferences.

Xiaoxue Wu is currently an associate professor of Yangzhou University. She received the Ph.D degree in Cyberspace Security from Northwestern Polytechnical University. Her main research direction is software quality assurance and testing. She has published more than 15 papers in prestigious journals of software testing.